



## ✓ Pneumonia Classification from Chest X-Ray Images

This notebook trains a ResNet18 model to classify chest X-rays into:

- BACTERIAL\_PNEUMONIA
- NORMAL
- VIRAL\_PNEUMONIA

### Instructions:

1. Go to Runtime > Change runtime type > Select GPU (T4)
2. Upload your dataset zip file when prompted
3. Run all cells

```
# Check GPU availability
import torch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
PyTorch version: 2.9.0+cu126
CUDA available: True
GPU: Tesla T4
```

```
# Upload and extract dataset
from google.colab import files
import zipfile
import os

print("Please upload your chest-x-ray-dataset-for-pneumonia-classification.zip file")
uploaded = files.upload()

# Extract the zip file
zip_filename = list(uploaded.keys())[0]
with zipfile.ZipFile(zip_filename, 'r') as zip_ref:
    zip_ref.extractall('/content/data')

print("\nExtracted contents:")
!ls -la /content/data/
```

Please upload your chest-x-ray-dataset-for-pneumonia-classification.zip file

chest-x-ray-dataset-for-pneumonia-classification.zip

**chest-x-ray-dataset-for-pneumonia-classification.zip**(application/zip) - 134477776 bytes, last modified: n/a - 100% done  
Saving chest-x-ray-dataset-for-pneumonia-classification.zip to chest-x-ray-dataset-for-pneumonia-classification.zip

```
Extracted contents:
total 16
drwxr-xr-x 4 root root 4096 Jan 14 23:00 .
drwxr-xr-x 1 root root 4096 Jan 14 23:00 ..
drwxr-xr-x 5 root root 4096 Jan 14 23:00 test
drwxr-xr-x 5 root root 4096 Jan 14 23:00 train
```

```
# Set data directories (adjust if your zip structure is different)
import os
```

```
# Find train and test directories
DATA_DIR = '/content/data'
for root, dirs, files in os.walk(DATA_DIR):
    if 'train' in dirs and 'test' in dirs:
        DATA_DIR = root
        break
```

```
TRAIN_DIR = os.path.join(DATA_DIR, 'train')
TEST_DIR = os.path.join(DATA_DIR, 'test')
```

```
print(f"Train directory: {TRAIN_DIR}")
```

```
print(f"Test directory: {TEST_DIR}")
print(f"\nClasses: {os.listdir(TRAIN_DIR)}")
```

```
Train directory: /content/data/train
Test directory: /content/data/test
```

```
Classes: ['VIRAL_PNEUMONIA', 'NORMAL', 'BACTERIAL_PNEUMONIA']
```

```
# Configuration
BATCH_SIZE = 32
NUM_EPOCHS = 10
LEARNING_RATE = 0.001
NUM_CLASSES = 3
IMAGE_SIZE = 224
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
# Imports and data loading
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
```

```
# Data transforms
train_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

```
test_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])
```

```
# Load datasets
train_dataset = datasets.ImageFolder(TRAIN_DIR, transform=train_transform)
test_dataset = datasets.ImageFolder(TEST_DIR, transform=test_transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                           shuffle=True, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
                          shuffle=False, num_workers=2, pin_memory=True)
```

```
classes = train_dataset.classes
print(f"Classes: {classes}")
print(f"Training samples: {len(train_dataset)}")
print(f"Test samples: {len(test_dataset)}")
```

```
Classes: ['BACTERIAL_PNEUMONIA', 'NORMAL', 'VIRAL_PNEUMONIA']
Training samples: 15618
Test samples: 618
```

```
# Create model
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
```

```
# Freeze early layers
for param in list(model.parameters())[: 101]:
```

```
for param in list(model.parameters())[:-10]:
    param.requires_grad = False
```

```
# Replace final layer
num_features = model.fc.in_features
model.fc = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(num_features, NUM_CLASSES)
)
```

```
model = model.to(device)
print("Model created: ResNet18 (pretrained, fine-tuned)")
```

Downloading: "<https://download.pytorch.org/models/resnet18-f37072fd.pth>" to /root/.cache/torch/100%[██████████] 44.7M/44.7M [00:00<00:00, 193MB/s]  
Model created: ResNet18 (pretrained, fine-tuned)

```
# Loss function with class weights for imbalanced data
class_counts = [7560, 4044, 4014] # BACTERIAL, NORMAL, VIRAL
weights = torch.FloatTensor([1.0/c for c in class_counts])
weights = weights / weights.sum() * len(class_counts)
criterion = nn.CrossEntropyLoss(weight=weights.to(device))
```

```
# Optimizer and scheduler
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
                        lr=LEARNING_RATE)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
```

```
# Training functions
def train_epoch(model, train_loader, criterion, optimizer):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(train_loader, desc="Training")
    for images, labels in pbar:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    pbar.set_postfix({
        'loss': f'{running_loss/total:.4f}',
        'acc': f'{100.*correct/total:.2f}%'
    })

    return running_loss / len(train_loader), 100. * correct / total
```

```
def evaluate(model, test_loader, criterion):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    all_predictions = []
    all_labels = []

    with torch.no_grad():
        for images, labels in tqdm(test_loader, desc="Evaluating"):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
```

```

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    all_predictions.extend(predicted.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())

accuracy = 100. * correct / total
avg_loss = running_loss / len(test_loader)

return avg_loss, accuracy, np.array(all_predictions), np.array(all_labels)

```

```

# Training loop
print("="*60)
print("Starting Training")
print("="*60)

train_losses, train_accs = [], []
val_losses, val_accs = [], []
best_acc = 0.0

for epoch in range(NUM_EPOCHS):
    print(f"\nEpoch {epoch+1}/{NUM_EPOCHS}")
    print("-" * 40)

    # Train
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer)
    train_losses.append(train_loss)
    train_accs.append(train_acc)

    # Evaluate
    val_loss, val_acc, _, _ = evaluate(model, test_loader, criterion)
    val_losses.append(val_loss)
    val_accs.append(val_acc)

    print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
    print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

    # Save best model
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'best_acc': best_acc,
            'classes': classes
        }, '/content/pneumonia_model.pth')
        print(f"Model saved (new best accuracy: {best_acc:.2f}%)")

    scheduler.step()

```

```

=====
Starting Training
=====

```

```
Epoch 1/10
```

```

-----
Training: 100%|██████████| 489/489 [00:51<00:00, 9.43it/s, loss=0.0168, acc=74.45%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 17.76it/s]
Train Loss: 0.5355, Train Acc: 74.45%
Val Loss: 1.1665, Val Acc: 70.55%
Model saved (new best accuracy: 70.55%)

```

```
Epoch 2/10
```

```

-----
Training: 100%|██████████| 489/489 [00:52<00:00, 9.39it/s, loss=0.0141, acc=78.10%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 17.78it/s]
Train Loss: 0.4493, Train Acc: 78.10%
Val Loss: 1.5158, Val Acc: 60.36%

```

```
Epoch 3/10
```

```
-----
Training: 100%|██████████| 489/489 [00:50<00:00, 9.62it/s, loss=0.0131, acc=79.93%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 18.01it/s]
Train Loss: 0.4181, Train Acc: 79.93%
Val Loss: 2.3300, Val Acc: 59.71%
```

Epoch 4/10

```
-----
Training: 100%|██████████| 489/489 [00:52<00:00, 9.32it/s, loss=0.0122, acc=81.23%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 11.73it/s]
Train Loss: 0.3898, Train Acc: 81.23%
Val Loss: 1.6273, Val Acc: 65.70%
```

Epoch 5/10

```
-----
Training: 100%|██████████| 489/489 [00:49<00:00, 9.88it/s, loss=0.0115, acc=82.10%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 11.91it/s]
Train Loss: 0.3677, Train Acc: 82.10%
Val Loss: 1.5675, Val Acc: 63.92%
```

Epoch 6/10

```
-----
Training: 100%|██████████| 489/489 [00:49<00:00, 9.89it/s, loss=0.0098, acc=85.29%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 11.40it/s]
Train Loss: 0.3132, Train Acc: 85.29%
Val Loss: 2.0990, Val Acc: 58.41%
```

Epoch 7/10

```
-----
Training: 100%|██████████| 489/489 [00:50<00:00, 9.76it/s, loss=0.0092, acc=86.09%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 12.35it/s]
Train Loss: 0.2952, Train Acc: 86.09%
Val Loss: 2.3112, Val Acc: 54.21%
```

Epoch 8/10

```
-----
Training: 100%|██████████| 489/489 [00:50<00:00, 9.75it/s, loss=0.0089, acc=86.51%]
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 14.76it/s]
```

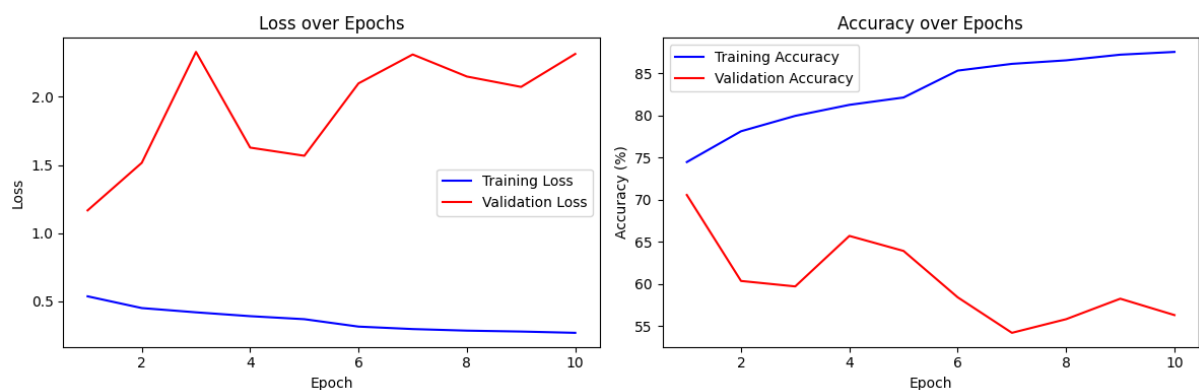
```
# Plot training history
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

epochs = range(1, len(train_losses) + 1)

ax1.plot(epochs, train_losses, 'b-', label='Training Loss')
ax1.plot(epochs, val_losses, 'r-', label='Validation Loss')
ax1.set_title('Loss over Epochs')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()

ax2.plot(epochs, train_accs, 'b-', label='Training Accuracy')
ax2.plot(epochs, val_accs, 'r-', label='Validation Accuracy')
ax2.set_title('Accuracy over Epochs')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy (%)')
ax2.legend()

plt.tight_layout()
plt.show()
```



```
# Final evaluation
print("="*60)
print("Final Evaluation on Test Set")
print("="*60)

checkpoint = torch.load('/content/pneumonia_model.pth')
model.load_state_dict(checkpoint['model_state_dict'])

_, final_acc, predictions, labels = evaluate(model, test_loader, criterion)

print(f"\nTest Accuracy: {final_acc:.2f}%")
print("\nClassification Report:")
print(classification_report(labels, predictions, target_names=classes))
```

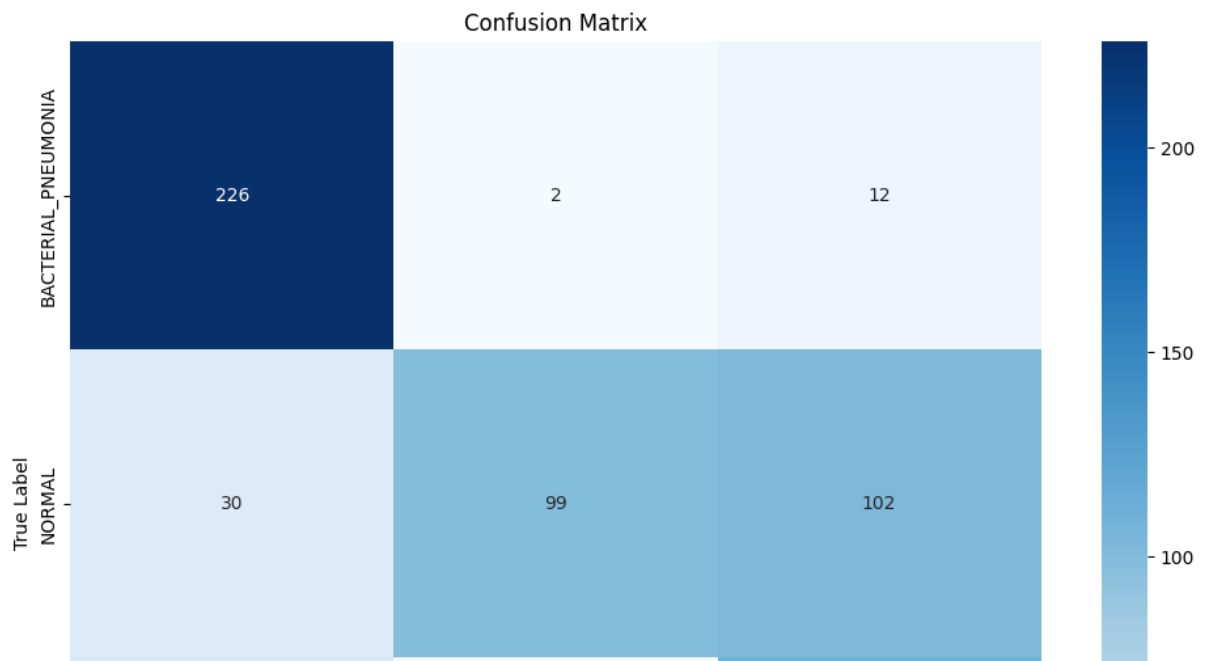
```
=====
Final Evaluation on Test Set
=====
Evaluating: 100%|██████████| 20/20 [00:01<00:00, 16.85it/s]
Test Accuracy: 70.55%

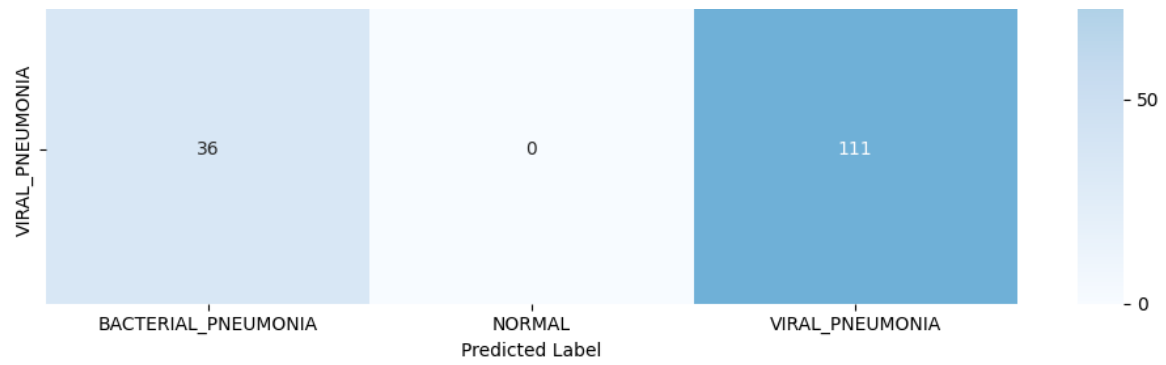
Classification Report:
              precision    recall  f1-score   support

BACTERIAL_PNEUMONIA      0.77      0.94      0.85        240
      NORMAL              0.98      0.43      0.60        231
      VIRAL_PNEUMONIA      0.49      0.76      0.60        147

   accuracy              0.71              618
  macro avg              0.75      0.71      0.68              618
 weighted avg              0.78      0.71      0.69              618
```

```
# Confusion matrix
cm = confusion_matrix(labels, predictions)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes)
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()
```





```
# Download the trained model
from google.colab import files
files.download('/content/pneumonia_model.pth')
print("Model downloaded! You can use this locally for inference.")
```

Model downloaded! You can use this locally for inference.